

Capítulo 18

Exploits

18.1. Objetivos

- Entender o que é um Buffer Overflow
- Aprender como explorar uma falha dessa categoria

18.2. Mas afinal, o que é um exploit?

Um exploit, em segurança da informação, é um programa de computador, uma porção de dados ou uma sequência de comandos que se aproveita das vulnerabilidades de um sistema computacional – como o próprio sistema operativo ou serviços de interação de protocolos (ex: servidores Web).

São geralmente elaborados por hackers como programas de demonstração das vulnerabilidades, a fim de que as falhas sejam corrigidas, ou por crackers a fim de ganhar acesso não autorizado a sistemas. Por isso muitos crackers não publicam seus exploits, conhecidos como 0days, e o seu uso massificado deve-se aos script-kiddies.

Quatro sites que podem ser usados como fonte de exploits são:



www.milw0rm.com

www.securityfocus.com

www.packetstormsecurity.com

www.metasploit.com

18.3. Organização dos Processos na Memória

Para entendermos como funciona um buffer overflow, nós precisaremos entender como funciona a pilha (stack).

A região de texto é fixa pelo programa e inclui as instruções propriamente ditas e os dados “somente leitura”. Esta região corresponde ao segmento de texto do binário executável e é normalmente marcada como somente-leitura para que qualquer tentativa de escrevê-la resulte em violação de segmentação (com o objetivo de não permitir código auto-modificável).

Os processos em execução são divididos em quatro regiões: texto, dados, pilha e heap. A pilha é um bloco de memória contíguo utilizado para armazenar as variáveis locais, passar parâmetros para funções e armazenar os valores de retornos

destas.

O endereço de base da pilha é fixo e o acesso à estrutura é realizado por meio das instruções PUSH e POP implementadas pelo processador. O registrador chamado "ponteiro de pilha" (SP) aponta para o topo da pilha.

A pilha consiste em uma seqüência de frames que são colocados no topo quando uma função é chamada e são retirados ao final da execução. Um frame contém os parâmetros para a função, suas variáveis locais, e os dados necessários para recuperar o frame anterior, incluindo o valor do ponteiro de instrução no momento da chamada de função.

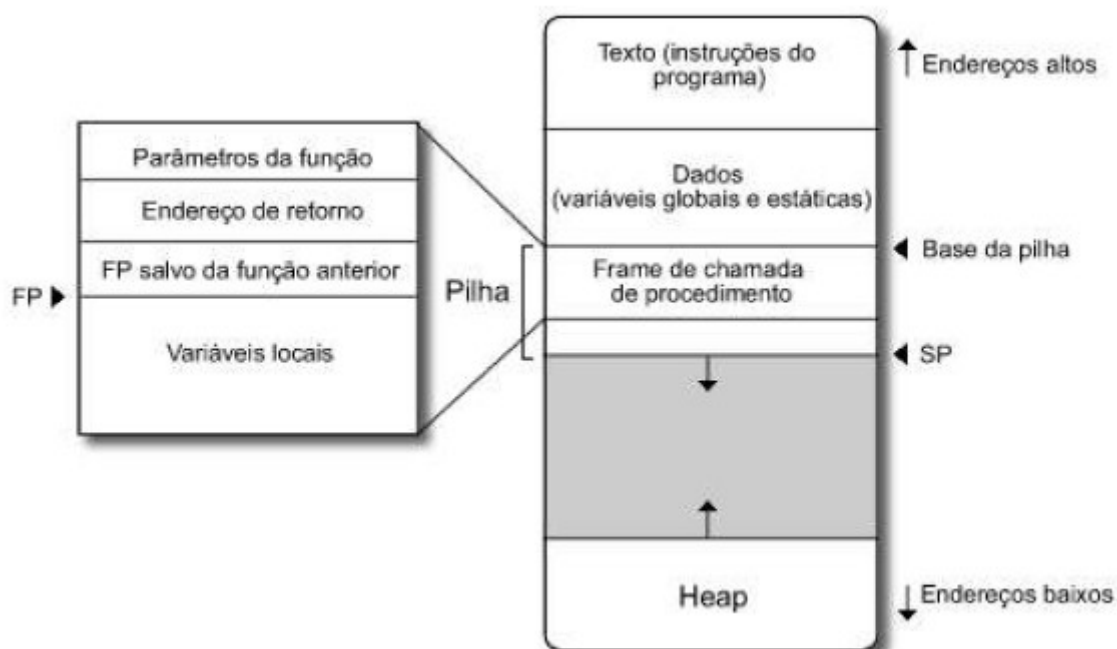
Dependendo da implementação, a pilha pode crescer em direção aos endereços altos ou baixos. O ponteiro de pilha também é de implementação dependente, podendo apontar para o último endereço ocupado na pilha ou para o próximo endereço livre. Como o texto trata da arquitetura Intel x86, iremos utilizar uma pilha que cresce para os endereços baixos, com o ponteiro de pilha (registrador ESP) apontando para o último endereço da pilha.

Além de um ponteiro de pilha, também é conveniente contar com um "ponteiro de frame" (FP) que aponta para um endereço fixo no frame. A princípio, variáveis locais podem ser referenciadas fornecendo-se seus deslocamentos em relação ao ponteiro de pilha. Entretanto, quando palavras são inseridas e retiradas da pilha, estes deslocamentos mudam. Apesar de em alguns casos o compilador poder corrigir os deslocamentos observando o número de palavras na pilha, essa gerência é cara. O acesso a variáveis locais a distâncias conhecidas do ponteiro de pilha também iria requerer múltiplas instruções. Desta forma, a maioria dos compiladores utiliza um segundo registrador que aponta para o topo da pilha no início da execução da função, para referenciar tanto variáveis locais como parâmetros, já que suas distâncias não se alteram em relação a este endereço com chamadas a PUSH e POP. Na arquitetura Intel x86, o registrador EBP é utilizado para esse propósito.

Por causa da disciplina de crescimento da pilha, parâmetros reais têm deslocamentos positivos e variáveis locais têm deslocamentos negativos a partir de FP.

A primeira instrução que um procedimento deve executar quando chamado é

salvar o FP anterior, para que possa ser restaurado ao fim da execução. A função então copia o registrador de ponteiro de pilha para FP para criar o novo ponteiro de frame e ajusta o ponteiro de pilha para reservar espaço para as variáveis locais. Este código é chamado de prólogo da função. Ao fim da execução, a pilha deve ser restaurada e a execução deve retomar na instrução seguinte à de chamada da função, o que chamamos de epílogo. As instruções CALL, LEAVE e RET nas máquinas Intel são fornecidas para parte do prólogo e epílogo em chamadas de função. A instrução CALL salva na pilha o endereço da instrução seguinte como endereço de retorno da função chamada. A instrução RET deve ser chamada dentro do procedimento e restaura a execução no endereço que está no topo da pilha.



18.4. Shellcode

Shellcode é um grupo de instruções assembly em formato de opcode para realizar diversas funções como chamar uma shell, ou escutar em uma porta. Geralmente, um shellcode é utilizado para explorar determinada vulnerabilidade, ganhando-se controle sobre a aplicação vulnerável e podendo-se executar qualquer instrução desejada.

Exemplo de shellcode mais simples possível:

Shellcode = “\xbb\x00\x00\x00\xb8\x01\x00\x00\xcd\x80”

O shellcode acima não é “injetável”, pois possui Null Bytes (/x00), o que caracteriza um final de string. Portanto, ao usarmos o shellcode acima, o programa encontrará o final da string e parará, não executando o restante do nosso payload. Mais detalhes veremos logo abaixo.

Shellcode Injetável: “\x31\xdb\xb0\x01\xcd\x80”

No exemplo de exploração de um Stack Overflow, utilizaremos um outro shellcode, que se encarregará de executar o /bin/sh ao invés de executar a função exit(), como o shellcode acima faz.

18.5. Buffer Overflow

Um buffer overflow acontece quando um programa vulnerável a esse tipo de falha tenta copiar mais informações para dentro de um buffer do que esse buffer consegue suportar. Para visualizar isso, é a mesma coisa que pegar uma garrafa de refrigerante de 2 litros e virar ela toda num copo de 500ml. Com certeza ocorrerá uma sujeira na mesa em que isso foi feito, e é a mesma coisa que ocorre na memória, um esparramado de caracteres sobre a memória que irá sobrescrever informações importantes assim como o refrigerante sujou toda a toalha da mesa.

As vulnerabilidades de buffer overflow são consideradas ameaças críticas de segurança, apesar de ser uma falha bem conhecida e bastante séria, que se origina exclusivamente na ignorância do programador referente a aspectos de segurança durante a implementação do programa, o erro se repete sistematicamente a cada nova versão ou produto liberado.

Este tipo de vulnerabilidade tem sido largamente utilizado para a penetração remota de computadores ligados a uma rede, onde um atacante anônimo tem como objetivo obter acesso ilegal ao computador vulnerável. Mesmo software considerado seguro, como o OpenSSH, já apresentou o problema, e também softwares famosos como o Sendmail e módulos do Apache.

Buffer overflows são também chamados de buffer overruns e existem diversos

tipos de ataques de estouro de buffer, entre eles stack smashing attacks, ataques contra buffers que se encontram na pilha (vou chamá-la de stack), e heap smashing attacks, que são ataques contra buffers que se encontram na heap. Tecnicamente, um buffer overflow é um problema com a lógica interna do programa, mas a exploração dessa falha pode levar a sérios prejuízos, como por exemplo, o primeiro grande incidente de segurança da Internet - o Morris Worm, em 1988 - utilizava técnicas de estouro de buffer, num programa conhecido como fingerd.

O objetivo de uma exploração contra um programa privilegiado vulnerável a buffer overflow é conseguir acesso de tal forma que o atacante consiga controlar o programa atacado, e se o programa possuir privilégios suficientes, ou seja se ele possui flag `suid root`, controlar a máquina.

Abaixo temos um pequeno programa vulnerável, que permitirá que façamos uma prova de conceito:



```
#include <stdio.h>

main() {
    char *name;
    char *dangerous_system_command;
    name = (char *) malloc(10);
    dangerous_system_command = (char *) malloc(128);
    printf("Address of name is %d\n", name);
    printf("Address of command is %d\n", dangerous_system_command);
    sprintf(dangerous_system_command, "echo %s", "Hello world!");
    printf("What's your name?");
    gets(name);
    system(dangerous_system_command);
}
```

Salve o arquivo como `overrun.c`, depois compile e execute-o com os seguintes comandos:



```
#gcc overrun.c -o over
#./over
```

Para testá-lo, digite o seguinte no prompt exibido pelo programa:



```
0123456789123456ls
```

E veja o que acontece!

O que causa essa falha, é o fato de que a variável “name” ocupa apenas um determinado espaço (que é pequeno) na memória. Quando estouramos esse espaço, e conseguimos sobrescrever a pilha EIP da memória, inserindo um comando que queiramos que seja executado, o resultado final é a execução do mesmo.

18.6. Conclusão

- Com isso, conseguimos demonstrar o perigo que é uma falha de programação em um programa.
- É possível obter controle completamente sobre o programa que esta sendo explorado.
- Se o programa oferecer algum serviço remotamente, a falha pode ser explorada remotamente, da mesma forma que foi explorada localmente, apenas trocando o shellcode e criando os sockets que serão responsáveis para se conectar no programa.